

Elev8ai MVP Build Plan

Sep 24, 2026 · @Sid

Overview

The MVP proves one loop: young people sign up, take paid jobs from your own companies, buy a course to unlock better jobs, and get paid. Target: a working product in about 8 weeks, built with Claude Code by you or one developer.

```
flowchart LR
    A[Sign up] --> B[Take open job]
    B --> C[Get paid]
    A --> D[Buy course]
    D --> E[Submit assignment]
    E --> F[Earn badge]
    F --> G[Unlock gated jobs]
    G --> C
```

The earning path and the learning path both end in paid work, which is the whole value of Elev8ai.

Launch companies (posted by admin): your agency (cold calling, content, clipping), your podcast (clipping), your clothing brand (content creation), and the web dev agency (small badge-gated dev tasks).

In the MVP:

- Talent signup, login and profile.
- Admin panel: create companies, post jobs, review applications, approve submissions, grade assignments, mark payouts.
- Job board with filters (category, pay, open vs badge-required).
- Job flow: apply, accept, submit proof, approve or request changes, get paid.
- Chat thread per job between talent and the company (you or your staff).
- In-app and email notifications.
- Course store: pay once, watch lessons, submit one assignment, get graded, earn a badge.
- Earnings page and payout onboarding.

Not in the MVP: self-serve company accounts, escrow, subscription tiers, agency seats, AI grading, ratings, mobile app, push notifications.

MVP revenue: course sales only. Jobs are funded by your own companies, so they are a cost of validating the model, not a revenue line yet.

Scope update: the items above marked "Not in the MVP" — self-serve company accounts, escrow, subscription tiers, agency seats, AI grading, ratings, mobile-friendly experience, and notifications — are now in scope for launch. See "Update: launch-ready scope expansion" at the end of this doc for the full detail, sequencing, and new Claude Code prompts. This section and the phase-by-phase plan above are left untouched as a historical record of what's already built; the update section is additive.

Accounts and services to create

You need about 14 accounts before or during the build; most are free until launch, and running costs at launch should be roughly \$60–120 per month plus payment fees. All prices below are approximate, so check each provider's pricing page when you sign up.

Payments warning for Mauritius: Stripe does not onboard Mauritius-registered businesses as platforms, and the usual workaround is forming a US LLC ([source](#)). For the MVP, avoid that complexity: sell courses through a merchant-of-record like Lemon Squeezy, and pay talent manually. Move to Stripe Connect in V2 once you have a supported entity.

#	Account	What it is for	Create by	Approx. cost
1	Domain (Cloudflare Registrar or Namecheap)	elev8ai domain for the app and email	Week 0	\$10–15/year
2	Business email (Google Workspace or Zoho Mail)	hello@, support@, payouts@ addresses	Week 0	Free (Zoho) to ~\$7/user/month
3	Claude subscription (Pro or Max)	Runs Claude Code, which writes the app	Week 0	See claude.com/pricing
4	GitHub	Stores the code, version history	Week 0	Free
5	Supabase	Database, login, file storage, realtime chat	Week 0	Free in dev, ~\$25/month at launch

#	Account	What it is for	Create by	Approx. cost
6	Vercel	Hosts the Next.js app	Week 0	Free in dev, ~\$20/month at launch (Hobby plan is non-commercial)
7	Resend	Sends notification and login emails from your domain	Week 1	Free tier covers the MVP
8	Lemon Squeezy (or MIPS / Paywise if buyers are mostly in Mauritius)	Takes course payments, handles sales tax	Week 5	~5% + \$0.50 per sale
9	Wise Business and/or PayPal Business	Paying international talent	Week 6	Per-transfer fees
10	Local bank / MCB Juice business	Paying talent in Mauritius	Week 6	Bank fees
11	Bunny Stream (or Mux)	Private course video hosting	Week 5	~\$1-10/month at MVP scale
12	Cal.com or Calendly	Booking links for cold-calling jobs, proof that meetings happened	Week 3	Free
13	Sentry	Error alerts when something breaks	Week 7	Free
14	PostHog	Product analytics and funnels	Week 7	Free tier

Non-technical setup:

- ☐ Register the business (Corporate and Business Registration Department in Mauritius) if not done through one of your existing companies.
- ☐ Check data-protection registration duties with the Mauritius Data Protection Office, since you store personal data.
- ☐ Terms of Service, Privacy Policy and Independent Contractor Agreement (a template tool like Termly to start, then a lawyer review).
- ☐ Instagram, TikTok and LinkedIn handles for Elev8ai to recruit talent.

- ☐ A password manager (Bitwarden or 1Password) shared with anyone helping you; turn on two-factor login on every account above.

Tech stack and architecture

One Next.js app on Vercel talks to Supabase for everything stateful; this is the stack Claude Code writes most reliably and one person can run.

Layer	Choice	Why
Frontend + backend	Next.js (App Router) + TypeScript	One codebase for pages and server logic
Styling	Tailwind CSS + shadcn/ui	Fast, consistent UI components
Database	Supabase Postgres	Relational data fits jobs, applications, badges
Auth	Supabase Auth (email magic link + Google)	No passwords to manage
Security	Row Level Security (RLS) policies	Talent only ever sees their own data
Files	Supabase Storage (private buckets)	Submissions, assignment uploads, avatars
Chat	Supabase Realtime	Live messages without a paid chat service
Email	Resend + React Email	Branded notification emails
Course video	Bunny Stream with signed URLs	Buyers-only playback
Course payments	Lemon Squeezy checkout + webhook	Purchase unlocks the course automatically
Payouts	Manual transfer, logged in admin	No payment-platform dependency in MVP
Validation	Zod	Checks every form and API input
Testing	Playwright	Tests the key flows end to end

```

flowchart LR
    U[Talent browser] --> N[Next.js on Vercel]
    A[Admin browser] --> N
    N --> S[(Supabase DB + Auth)]
    N --> F[Supabase Storage]
    N --> R[Resend email]
    L[Lemon Squeezy] -- webhook --> N
    N --> B[Bunny video]

```

Every request goes through the Next.js app, and only Lemon Squeezy calls in from outside, through a signed webhook.

Two roles in the MVP: `talent` (default on signup) and `admin` (you and your staff, set by hand in the database). Companies are records the admin manages, not user accounts yet.

Database schema

Sixteen tables cover the whole MVP; money is stored in cents as integers, and every table has `id` (uuid) and `created_at`.

Table	Key fields	Notes
profiles	user_id, role (talent/admin), full_name, username, avatar_url, country, date_of_birth, bio, phone	One per auth user; signup blocked under 18
payout_details	user_id, method (bank/juice/wise/paypal), details (json)	Readable only by the owner and admins
companies	name, slug, logo_url, description, website	Admin-managed; your 4 launch companies
skills	slug, name, description	e.g. clipping, cold_calling, content, web_dev
user_skills	user_id, skill_id, source (course/manual), awarded_by	These are the badges
jobs	company_id, title, description, category, pay_cents, pay_type (fixed/per_unit), unit_label, max_units, slots, required_skill_id (nullable), deadline, proof_instructions, status (draft/open/closed)	required_skill_id empty = open to all
applications	job_id, user_id, pitch, status (pending/accepted/rejected/withdrawn)	Unique per job + user
submissions	application_id, notes, links[], file_paths[], units_claimed, status (submitted/changes_requested/approved/rejected), reviewer_note	Proof of work

Table	Key fields	Notes
payouts	user_id, submission_id, amount_cents, status (owed/paid), method, reference, paid_at	Created on approval, marked paid by admin
conversations	application_id	One chat thread per accepted job
messages	conversation_id, sender_id, body, attachment_path, read_at	Realtime enabled
notifications	user_id, type, title, body, link, read_at, emailed_at	Bell icon + email
courses	slug, title, description, price_cents, skill_id, lemon_variant_id, published	Passing awards skill_id
lessons	course_id, position, title, body_md, video_id	Ordered lessons
course_access	user_id, course_id, order_id, amount_cents	Created by payment webhook
course_assignments	user_id, course_id, notes, links[], file_paths[], status (pending/passed/failed), feedback, graded_by	Manual grading

A job moves through these states for each worker:

```
stateDiagram-v2
    [*] --> Applied
    Applied --> Accepted
    Applied --> Rejected
    Accepted --> Submitted
    Submitted --> ChangesRequested
    ChangesRequested --> Submitted
    Submitted --> Approved
    Submitted --> RejectedWork
    Approved --> PayoutOwed
    PayoutOwed --> Paid
    Paid --> [*]
```

Every arrow sends a notification to the other side, which is how talent always knows where their money is.

Build timeline

Eight weeks from empty repo to public launch, working in seven build phases; each phase ends with something you can click through and test.

Week	Phase	Done when
0	Setup: accounts, Claude Code, repo, first course outlines	Every account in the table above exists; blank app deploys to Vercel
1	1. Foundation: auth, profiles, roles, app layout	You can sign up, finish onboarding, and see an empty dashboard; admin sees an admin area
2	2. Companies and jobs: admin posting, talent job board	Admin posts a clipping job for the podcast; talent sees and filters it
3	3. Job workflow: apply, accept, submit, review, payout ledger	A test user completes a job end to end and a payout shows as owed
4	4. Chat and notifications	Admin and talent message live; each status change sends in-app + email alerts
5	5. Courses: catalog, lessons, video, checkout	Paying for Clipping 101 in test mode unlocks its lessons
6	6. Assignments, badges, gated jobs, payout details	Passing an assignment awards a badge that unlocks a cold-calling job
7	7. Launch prep: landing page, legal pages, analytics, security review, tests	Every item in the launch checklist is ticked
8	Private beta (20–30 talent), fixes, public launch	First real payouts sent; first course sales

If you fall behind, cut from week 7 polish, never from the payout flow or the badge loop.

Claude Code setup

Install Claude Code, give it a CLAUDE.md that describes Elev8ai, then feed it the phase prompts one at a time; setup takes about an hour. Claude Code needs a Claude subscription or an Anthropic Console account ([docs](#)).

1. Install the tools on your computer

- Node.js (current LTS), Git, and VS Code.
- Claude Code, using the native installer from the official docs:

```
# macOS, Linux, WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex
```

Prefer not to use a terminal? The Claude Desktop app also runs Claude Code.

2. Create the project

```
mkdir elev8ai && cd elev8ai
git init
claude
```

You log in on first launch. Then create the CLAUDE.md below in the project root before running any phase prompt.

3. CLAUDE.md (paste as-is, edit as the project evolves)

```
# Elev8ai

Marketplace where young people (18+) take paid micro-jobs from companies
and buy courses that unlock skill-gated jobs.

## MVP scope
- Roles: talent (default) and admin. Companies are admin-managed records.
- Job categories: clipping, cold_calling, content, web_dev.
- Flow: apply -> accept -> submit proof -> approve -> payout owed -> paid.
- Courses: pay once (Lemon Squeezy), lessons, one assignment,
  manual grading, pass awards a skill badge that unlocks gated jobs.
- Chat per accepted application. In-app + email notifications.
- Payouts are manual transfers logged by admin. No escrow, no subscriptions.

## Stack
Next.js App Router, TypeScript strict, Tailwind, shadcn/ui, Supabase
(Postgres, Auth, Storage, Realtime), Resend, Zod, Playwright.

## Rules
- Money is integer cents. Never floats.
- Every table has RLS enabled. Talent can only read/write their own rows.
  Admin checks go through a SQL function is_admin().
- All DB changes are SQL migration files in supabase/migrations.
- Validate every form and server action input with Zod.
- Server-only secrets never reach client components.
- Mobile-first UI; most talent will use phones.
- Small, reviewable changes. Explain the plan before large edits.
```

- After each feature: run typecheck, lint, and relevant tests.

Commands

```
npm run dev | npm run build | npm run lint | npm run typecheck | npm run test:e2e
```

4. How to work with it

- One prompt from the next section at a time. Each starts by asking for a plan; read it, correct it, then let it build.
- Commit to GitHub after every working step, so you can always roll back.
- Start a fresh Claude Code session at the start of each phase so it works from CLAUDE.md and the code, not a long old conversation.
- Put secrets only in `.env.local` (never committed) and in Vercel's environment settings. Never paste keys into prompts.
- Test every flow yourself on your phone before moving on. When something breaks, paste the exact error and what you did.

Claude Code prompts

Run these in order, one per session step, and commit after each works. Each prompt assumes CLAUDE.md is in place. Before Phase 2, copy the Database schema table from this doc into docs/schema.md in the repo, because several prompts point Claude Code to it.

Phase 0: Scaffold

```
Read CLAUDE.md. Show me a plan first, then scaffold the project:
- Next.js App Router, TypeScript strict, Tailwind, shadcn/ui, ESLint, Prettier.
- Scripts: dev, build, lint, typecheck, test:e2e (Playwright installed).
- Supabase: init the supabase/ folder for migrations, add @supabase/ssr helpers for browser client, server client, and middleware session refresh.
- .env.example listing every variable we will need (Supabase URL, anon key, service role key, Resend key, Lemon Squeezy webhook secret, Bunny keys, app URL) with a comment on each. Make sure .env.local is gitignored.
- A placeholder home page that says Elev8ai.
Run build and typecheck and fix anything that fails.
```

Phase 1: Foundation

1a. Profiles and roles

Plan first. Create a migration for:

- profiles (user_id pk references auth.users, role text check in ('talent','admin') default 'talent', full_name, username unique, avatar_url, country, date_of_birth, phone, bio, onboarded boolean).
- A trigger that inserts a profiles row when a new auth user is created.
- A SQL function is_admin() returning true if the current user's role is admin.
- RLS: users read and update only their own profile and cannot change their own role; admins read all profiles.
- A public avatars storage bucket with policies so users can only write to their own folder.

Generate TypeScript types from the schema and explain how I apply the migration to my Supabase project.

1b. Auth and onboarding

Plan first. Build authentication:

- /login with email magic link and Google sign-in via Supabase Auth.
- Auth callback route and middleware that protects /app/** and /admin/**.
- /onboarding form (full name, username, date of birth, country, phone, short bio, avatar upload). Reject anyone under 18 with a friendly message. Validate with Zod in a server action.
- After login: not onboarded -> /onboarding; admin -> /admin; else /app/jobs.
- Logout button.

1c. App shell

Plan first. Build the layouts:

- Talent area (/app): mobile-first, bottom navigation on phones and a sidebar on desktop with Jobs, My Jobs, Learn, Messages, Profile, plus a notification bell in the header (placeholder for now).
 - Admin area (/admin): sidebar with Companies, Jobs, Applications, Submissions, Payouts, Courses, Grading, Users. Only admins can open it, checked on the server.
 - Clean, modern style: dark navy and electric-green accent, rounded cards.
- Add empty placeholder pages for each nav item.

Phase 2: Companies and jobs

2a. Schema

Plan first. Migration for companies, skills, user_skills and jobs exactly as described in docs/schema.md (jobs: company_id, title, description, category in clipping/cold_calling/content/web_dev, pay_cents int, pay_type in fixed/per_unit, unit_label, max_units, slots, required_skill_id nullable, deadline, proof_instructions, status in draft/open/closed).

RLS: anyone logged in can read companies, skills, and jobs with status open; only admins can write. user_skills: users read their own, admins manage all. Seed 4 skills (clipping, cold_calling, content, web_dev) and 4 companies (my agency, my podcast, my clothing brand, the web dev agency) with placeholder descriptions I can edit.

2b. Admin posting

Plan first. Build admin pages:

- Companies: list, create, edit, logo upload.
- Jobs: list with filters by company/status/category; create and edit form with every job field; required skill picker ('Open to everyone' or a skill); save as draft or publish; close a job; duplicate a job.
- Show pay nicely (e.g. \$5 per clip, max 20 clips).

2c. Talent job board

Plan first. Build /app/jobs for talent:

- Cards with company logo, title, pay, category, deadline, spots left.
- Filters: category, minimum pay, 'jobs I can take now' vs 'locked'.
- Locked jobs (required skill the user lacks) show a lock and a link to the course that awards that skill (leave the link to /app/learn for now).
- Job detail page with full description and proof instructions.
- Empty states and loading skeletons. Works well on a phone.

Phase 3: Job workflow

3a. Schema

Plan first. Migration for applications, submissions and payouts (fields as in docs/schema.md: applications unique per job+user; submissions with links[],

file_paths[], units_claimed, status, reviewer_note; payouts with amount_cents, status owed/paid, method, reference, paid_at).

Private 'submissions' storage bucket: users write only to their own folder, admins read all. RLS: talent see only their own applications, submissions and payouts; admins see all.

3b. Apply and accept

Plan first. Build:

- Apply button with optional short pitch on the job detail page.
On the server, block: already applied, job not open, no slots left, missing required skill. Show clear reasons.
- /app/my-jobs with tabs: Applied, In progress, Submitted, Completed.
- /admin/applications queue: accept or reject with optional note.
Accepting counts against slots; auto-close the job when slots are full.

3c. Submit, review, payout ledger

Plan first. Build:

- Submission form on an accepted job: links, file uploads, units claimed (for per_unit jobs), notes. Show the job's proof instructions above it.
- /admin/submissions review queue with approve, request changes (note required), reject (note required).
- Approval runs in one Postgres function (transaction): mark approved and create a payout row owed = pay_cents for fixed jobs, or $\text{pay_cents} \times \min(\text{units approved}, \text{max_units})$ for per_unit jobs. Admin can adjust approved units.
- /app/earnings: owed, paid, lifetime total, and history.
- /admin/payouts: list owed payouts grouped by user; mark paid with method and reference; CSV export.

Add a Playwright test for apply -> accept -> submit -> approve -> paid.

Phase 4: Chat and notifications

4a. Chat

Plan first. Build per-job chat with Supabase Realtime:

- Migration: conversations (one per accepted application, created automatically on acceptance) and messages (sender_id, body, attachment_path, read_at). RLS: only the application's talent and admins can read or write. Enable realtime on messages.
- /app/messages inbox and /admin/messages inbox: threads sorted by latest message, unread counts, job title shown on each thread.
- Thread view: live updates, image/file attachments (private bucket), mark as read on open, 'Open job' link.
- Keep people from sharing phone numbers or emails in chat for now by showing a gentle warning if a message contains one (do not block).

4b. Notifications

Plan first. Build notifications:

- Migration for notifications (as in docs/schema.md) plus an `email_opt_out` boolean on profiles. RLS: users see only their own.
- A server helper `notify(userId, type, title, body, link)` that inserts the row and, unless opted out, sends an email through Resend with a React Email template in Elev8ai branding.
- Fire it on: application accepted/rejected, submission approved/changes requested/rejected, payout marked paid, new message (email at most once per thread per hour), new job posted in a category the user has worked in, course purchased, assignment graded. Admins get: new application, new submission, new assignment to grade.
- Header bell with live unread count and a dropdown of the latest 10; `/app/notifications` full list; mark all as read.

Phase 5: Courses

5a. Course admin

Plan first. Migration for courses, lessons and `course_access` (see docs/schema.md). RLS: anyone logged in reads published courses and lesson titles; lesson bodies and video ids only for users with `course_access` or admins. Admin pages: create/edit course (title, description, price, skill awarded, Lemon Squeezy variant id, published), lesson editor with ordering, markdown body with preview, and Bunny video id.

5b. Learning pages

Plan first. Build `/app/learn`:

- Catalog of published courses with price and the jobs each unlocks.
- Course page: syllabus, price, 'Buy' button, or 'Continue' if owned.
- Lesson page for owners: markdown body and a Bunny Stream player using a signed embed URL generated on the server that expires after a few hours.
- Mark lesson complete, progress bar, next/previous lesson.
- Replace the job-board 'locked' link so it points at the right course.

5c. Payments

Plan first. Integrate Lemon Squeezy:

- 'Buy' opens the Lemon Squeezy checkout for the course's variant, passing the user id as custom data and prefilling their email.
 - Route `/api/webhooks/lemonsqueezy` verifies the signature with the webhook secret, handles `order_created`, is idempotent on order id, inserts `course_access`, and calls `notify()`. Handle refunds by removing access.
 - A success page that waits for access to appear, then opens lesson 1.
- Explain how to test this in Lemon Squeezy test mode locally.

Phase 6: Badges and gated jobs

6a. Assignments and grading

Plan first. Build course assignments:

- Add an `assignment_brief` field to courses. After the last lesson, show the brief and a submission form (links, files, notes). One pending submission at a time; resubmission allowed after a fail.
- `/admin/grading queue`: view submission, pass or fail with required feedback. Pass inserts the course's skill into `user_skills` in the same transaction. Notify the user either way.
- Show earned badges on the profile and next to the user's name in admin.
- Admin can also award or revoke a badge manually from the Users page.

6b. Unlock moment and payout details

Plan first. Build:

- When a badge is awarded, notify the user with a link to the job board filtered to jobs that badge unlocks; show a 'Newly unlocked' tag on those jobs for 7 days.
- `/app/settings/payout`: choose method (bank transfer, MCB Juice, Wise, PayPal) and enter details. Stored in `payout_details`, readable only by the owner and admins. Talent must fill this before their first payout.
- Show each user's payout details on `/admin/payouts` next to owed amounts.

Phase 7: Launch prep

7a. Public pages

Plan first. Build the public site:

- Landing page: hero ('Get paid for real work. Learn skills that pay more.'), how it works in 3 steps, job categories, featured courses, launch companies, FAQ, sign-up call to action.
- `/terms`, `/privacy` and `/contractor-agreement` pages that render markdown files from `/content` so I can paste in final legal text.
- SEO metadata, Open Graph image, sitemap, robots.txt, favicon.

7b. Security review

Act as a strict security reviewer. Audit every RLS policy, storage policy, server action, API route and webhook. Try to find ways a talent user could read another user's data, approve their own submission, change their role, grant themselves course access or a badge, or see payout details. List findings by severity with file paths, then fix them one by one and add tests that prove each fix.

7c. Tests, monitoring, production

Plan first. Then:

- Playwright tests for: signup + onboarding, under-18 rejection, job apply-to-paid flow, gated job blocked then unlocked after passing an assignment, course purchase via a mocked webhook, chat message delivery.
- Add Sentry error tracking and PostHog analytics with events: signed_up, onboarded, job_applied, job_accepted, submission_approved, payout_paid, course_viewed, course_purchased, assignment_passed.
- Write docs/deploy.md: every production environment variable, creating a separate production Supabase project, applying migrations, verifying the Resend domain, switching Lemon Squeezy to live mode and the live webhook.

Bug-fix template

Something is broken. What I did: <steps>. What I expected: <expected>. What happened: <actual>. Exact error: <paste>. Find the root cause before changing code, explain it in two sentences, fix it, and add a test so it cannot come back.

Content and operations prep

While the app is being built, prepare the courses, the first jobs and the payout routine in parallel; without them the finished app launches empty.

Courses (record by week 5)

Course	Price	Lessons	Assignment	Pass criteria	Unlocks
Clipping 101	~\$12	5–6 short videos: finding the hook, cutting, captions, formats per platform, posting	3 clips from a podcast episode you provide	Hook in first 3 seconds, clean captions, correct format, on-brand	Paid podcast and agency clipping jobs
Cold Calling & Appointment Setting	~\$39	6–8 videos: prospect research, openers, objection handling, booking, CRM notes	Recorded 3–5 minute mock call plus a call script	Rubric score of 7/10 or more	Cold-calling jobs
Content Creation (optional at launch)	~\$19	5 videos: shooting product content, hooks, editing, brand guidelines	2 short videos for the clothing brand	Usable as-is by the brand	Advanced content jobs

First jobs (ready by week 7)

- ☐ Write 10–15 job posts across the four companies, with exact proof instructions for each.
- ☐ Open starter jobs (no badge): simple content ideas, basic clips at \$1–5 each, so free users earn quickly.
- ☐ Gated jobs: podcast clips at a higher rate, cold-calling at a set price per meeting held, one small web dev task.
- ☐ Decide each company's monthly job budget so payouts never surprise you.

Operating rules

Process	Rule
Review submissions	Within 48 hours
Grade assignments	Within 72 hours, with written feedback
Payouts	Weekly, every Friday, for everything approved by Thursday
Minimum payout	\$10, to keep transfer fees sensible
Cold-calling proof	Meeting counts only once it actually takes place
Clipping proof	Posted link plus a views screenshot after 48–72 hours
Support	support@ email answered within 1 business day

Beta recruitment (weeks 6–8)

- ☐ Recruit 20–30 beta users from your networks, your podcast audience and local universities.
- ☐ Give beta users Clipping 101 free in exchange for feedback.
- ☐ Final legal text for Terms, Privacy Policy and Independent Contractor Agreement, reviewed by a lawyer.

Testing and launch checklist

Launch only when every box below is ticked; the security and money items are non-negotiable.

Manual test run (on a phone and a laptop)

- ☐ New user signs up, is rejected if under 18, completes onboarding.

- ☐ Talent applies to an open job, is accepted, chats with admin, submits proof, gets approved, sees payout owed, then paid.
- ☐ Gated job cannot be applied to without the badge, even by editing the URL.
- ☐ Talent buys a course in test mode, watches lessons, submits the assignment, passes, and can now apply to the gated job.
- ☐ Every status change produced an in-app notification and an email.
- ☐ A second test user cannot see the first user's applications, submissions, messages, payouts or payout details.

Security and money

- ☐ Security review prompt (7b) run and every finding fixed.
- ☐ RLS enabled on every table (check in the Supabase dashboard).
- ☐ Service role key used only on the server; not in any client bundle.
- ☐ Webhook signature verified; replaying the same order does not grant access twice.
- ☐ Supabase daily backups on for the production project.

Production setup

- ☐ Separate production Supabase project with all migrations applied.
- ☐ Production environment variables set in Vercel; custom domain connected with HTTPS.
- ☐ Resend domain verified (SPF/DKIM) so emails don't land in spam.
- ☐ Lemon Squeezy in live mode with the live webhook; one real \$1 test purchase refunded.
- ☐ Sentry and PostHog receiving events from production.
- ☐ Final legal pages published; admin roles set for you and your staff.

Launch day

- ☐ 10-15 jobs open, at least 2 courses published.
- ☐ Announce on your podcast, agency and brand channels.
- ☐ Check error alerts and the review queues twice a day for the first two weeks.

Metrics to track after launch

Review these weekly for the first 8 weeks; they decide whether to build V2 (subscriptions and outside companies) and where to set tier limits. Targets are starting guesses to revise with real data.

Metric	Starting target	What a miss tells you
Signup to onboarded	70%	Onboarding is too long
Onboarded to first application within 7 days	50%	Not enough open starter jobs
Accepted jobs completed and approved	70%	Jobs unclear or proof too hard
Median submission review time	Under 48 hours	Your team is the bottleneck
Course page view to purchase	5-10%	Price or the unlock promise is weak
Buyers who submit the assignment	60%	Course too long or brief unclear
Assignment pass rate	50-70%	Too low: course weak; too high: bar too easy
Badge holders who take a gated job within 14 days	60%	Not enough gated jobs to reward the purchase
Talent active again after 30 days	30%	Not enough ongoing work
Course revenue vs job payouts	Track the ratio	How much the marketplace costs you to run

The single most important number is the badge-to-gated-job rate: if people who pay to learn quickly earn more, the whole Elev8ai model works.

Update: paid since launch, no free tier

Decision: the platform launches paid from day one. Browsing is free; applying is not.

flowchart TD

```

A[Visitor browses job board] --> B{Logged in?}
B -- No --> C[Sign up / log in]
B -- Yes --> D{Active paid plan?}
C --> D
D -- No --> E[Plan picker: choose a plan to apply]
D -- Yes --> F[Apply to job]
E --> G[Subscribe via Mobipaid]
G --> F

```

Job listings, company profiles, and course previews stay publicly visible with no login required — that's the marketing surface. The gate sits specifically at the Apply action: submitting an application server-side checks for an active subscription, and if none exists, redirects to plan selection instead of letting the request through. Never gate this client-side only; the server action must re-check, since a client-side-only gate is trivially bypassed.

Revised tiers (Free tier removed):

	Starter	Pro	Mini-Agency
Price	~\$5/month	~\$15/month	~\$59/month
Max job value	\$25	\$250	Unlimited + agency-only jobs
Active jobs at once	3	5	20 (shared across seats)
Talent-side fee on earnings	8%	5%	3%
Course discount	—	20%	30%
Seats	1	1	3

Starter replaces the old Free tier as the entry point — low enough to not be a real barrier for someone about to earn \$20–50 on their first job, but real enough that every signed-up applicant is a paying customer from day one. Re-test this price once you have real signup data; if Starter conversion is weak, it's the first lever to pull.

Update: escrow on accept (Fix 1) — required once outside companies join

This closes the "download the work, reject it, pay nothing" risk. The money moves on acceptance, not on approval.

flowchart TD

```

A[Company accepts application] --> B[Charge company, hold in escrow]
B --> C[Talent submits proof]
C --> D[Company reviews]
D -- Approved --> E[Escrow releases to talent]
D -- Changes requested, max 2 rounds --> C
D -- Rejected --> F[Status: disputed, escrow stays frozen]
F --> G[Talent responds within 48h]
G --> H[Admin arbitrates against proof_instructions]
H -- Talent wins --> E
H -- Company wins --> I[Escrow refunds to company]

```

Schema changes this implies (plan now, build when escrow is added):

- `applications`: add `escrow_status` (`pending`, `held`, `released`, `refunded`).
- `submissions`: add `disputed` to the status enum, alongside `approved/rejected/changes_requested`.
- `disputes` table: `submission_id`, `talent_response`, `admin_decision`, `decided_by`, `decided_at`.
- File-based jobs (clips, documents, code): the delivered file stays watermarked or reduced-quality in the submission record until escrow releases; only then does the clean file unlock. This is the part that actually stops "download and reject" — escrow alone doesn't, since a company can still see a full-quality file before paying unless delivery itself is gated.

Trust guardrails to track from day one, even while all jobs are still your own companies: rejection/dispute rate per company, visible on the admin dashboard, with a manual-review flag once a company crosses roughly 30% rejections, and a "small jobs only" cap (e.g. under \$20) for any company until they've built a clean track record.

Update: talent profile, LinkedIn-style layout

Replaces the minimal MVP profile with a fuller one companies will actually judge someone by. New fields on `profiles` (or a linked `portfolio_items` table where noted):

Section	Fields	Notes
Header	Profile photo, cover banner (optional), full name, headline (e.g. "Clipper Cold Caller")	Headline is a single short line, editable, shown on every card
About	Short bio (plain text, ~500 chars)	
Badges	Pulled from <code>user_skills</code>	Shown as a row of icons/chips near the top, not buried
Portfolio	New <code>portfolio_items</code> table: title, description, media (image/video/link), date	Lets talent upload sample clips, content, or work even before their first platform job — important for someone with zero job history yet
Work history	Auto-populated from completed, approved jobs	Company name, job title, completion date — no pay amount shown publicly

Section	Fields	Notes
Stats	Jobs completed, badges earned, member since	Public, builds credibility at a glance

What's public vs. private: photo, headline, bio, badges, portfolio, and work history are public to any logged-in user (companies browsing talent later, other talent). Pay amounts, payout details, and private messages stay private regardless of profile visibility settings.

Security checklist

Handling bank details, payment data, and minors' personal information means security isn't optional polish — treat every item below as a launch blocker, not a nice-to-have.

Authentication & access

- ☐ Supabase Auth with magic link + Google OAuth only — no self-managed passwords to leak.
- ☐ Rate-limit login and signup attempts (Supabase has built-in limits; verify they're on).
- ☐ Admin accounts require 2FA, enforced, not optional.
- ☐ Every table has RLS enabled — checked directly in the Supabase dashboard, not assumed from migration files.
- ☐ Admin checks go through a single `is_admin()` SQL function, never a client-side role check.
- ☐ Session cookies are httpOnly and secure; verify Supabase's SSR helpers are configured this way.

Input handling

- ☐ Every form and server action validated with Zod — no raw user input reaches the database or a file path.
- ☐ File uploads restricted by type and size at both the client and server; stored in private buckets, never public.
- ☐ User-generated content (bios, messages, job descriptions) sanitized before rendering, to block stored XSS.
- ☐ A Content-Security-Policy header set on the app, restricting script sources.

Secrets & infrastructure

- ☐ Service role keys and webhook secrets live only in Vercel's environment settings — never in client bundles, never committed to Git.
- ☐ `.env.local` gitignored from day one; scan the repo history for leaked secrets before going public.

- ☐ HTTPS enforced everywhere (Vercel does this by default) with HSTS enabled.
- ☐ Webhook signatures (Lemon Squeezy, Mobipaid) verified on every request; replayed webhooks must be idempotent, not double-process a payment.

Payment & financial data

- ☐ Never store raw card numbers — all card handling stays inside Lemon Squeezy/Mobipaid's hosted checkout, never touching your server.
- ☐ Bank account numbers and beneficiary names (payout_details) encrypted at rest or stored in a column with strict RLS, visible only to the owning user and admins.
- ☐ Payout details locked from editing for 24 hours after a payout is marked owed, to prevent last-minute account-swap fraud.
- ☐ **QR code integrity (Juice payments)**
 - ☐ The talent's QR is fetched fresh from the database on every render, scoped to that specific job's application id — never cached or reused across jobs in the same session.
 - ☐ The payment screen shows talent's name and the last few digits of their registered phone/account next to the QR, so the company can visually confirm before scanning.
 - ☐ QR/payout detail changes trigger the same 24-hour edit lock and owner notification as other payout details.
 - ☐ Every QR display event is logged (job id, talent id, company id, timestamp), so any misdirected payment has a full audit trail.
 - ☐ The QR is only revealed after submission approval, never earlier, to minimize exposure window.

Multi-tenancy and data isolation

- ☐ An automated cross-tenant test suite: for every table, log in as User A and User B and assert B cannot read, write, or enumerate A's rows (applications, submissions, messages, payout details, company data, disputes).
- ☐ This suite re-runs on every new table added going forward — a standing CLAUDE.md rule, not a one-time audit.
- ☐ Every query is explicitly scoped by owner in code, never relying on the UI alone to hide rows the API would still return.
- ☐ Foreign key constraints make it structurally impossible for a submission to attach to the wrong job or applicant.

Hardening beyond the basics

- ☐ A focused third-party penetration test on auth, RLS policies, and payment-adjacent endpoints before public launch.

- ☐ Rate limiting on every public endpoint, not just login — job applications, submissions, and messages included.
- ☐ A WAF in front of the app (Cloudflare's free tier covers much of this).
- ☐ The service role key is used only in the specific server actions that truly need to bypass RLS, never as a default.
- ☐ Malware scanning on uploaded files, since talent upload files directly into storage.

Reliability and crash prevention

- ☐ Heavy work (watermark generation, file processing) runs in a background job queue, async with retries — never inline in a web request.
- ☐ Idempotency and retry-with-backoff on every external call (Calendly, MIPS, any future API).
- ☐ Load testing before launch, simulating a realistic signup spike.
- ☐ A feature kill-switch for new, risky features (e.g., a new job category's verification logic) so one feature can be disabled without taking down the whole app.
- ☐ Uptime monitoring with alerts, separate from Sentry's error tracking.
- ☐ A database restore has actually been tested, not just assumed to work because backups are enabled.

Monitoring & response

- ☐ Sentry capturing errors in production, with alerts routed to you, not just a dashboard no one checks.
- ☐ Admin actions (approvals, rejections, manual payouts, role changes) logged with who-did-what-when, queryable later if a dispute arises.
- ☐ Supabase daily backups enabled on the production project, and you've actually tested a restore at least once.

Compliance

- ☐ Mauritius Data Protection Act obligations checked with the Data Protection Office, since the app stores personal data including minors' ages and bank details.
- ☐ Privacy Policy discloses exactly what's collected and why, written in plain language, not boilerplate.
- ☐ Under-18 signups blocked server-side, not just with a client-side date picker restriction.

Ongoing maintenance (don't treat security as a one-time launch task)

Cadence	Task
Weekly	Review Sentry errors and Supabase logs for anomalies
Weekly	Check for dependency vulnerability alerts (GitHub Dependabot, <code>npm audit</code>)
Monthly	Update dependencies, especially Next.js, Supabase client libraries, and auth-related packages
Quarterly	Re-run the security review prompt (7b) against the current codebase, since new features open new surface area
Quarterly	Review admin access list — remove anyone who no longer needs it
On every new integration	Re-check webhook signature verification and RLS policies for any new table

Ask Claude Code to run this as a recurring task: "Run `npm audit` and list any high or critical vulnerabilities, then check Dependabot alerts on GitHub and summarize what needs updating."

MVP scope for this: profile photo upload, headline, bio, badges, and the portfolio table are in scope now. Auto-populated work history can land as soon as the job workflow (Phase 3) is live, since it just reads from existing `applications / submissions` data.

Deferred to a later version

Companies browsing talent and assigning tasks directly (viewing profiles and inviting a specific person to a job, rather than only posting publicly and waiting for applications) is explicitly out of MVP scope, even though the LinkedIn-style profile groundwork above makes it easy to add later. It needs a direct-invite flow and company-side search/filter UI that isn't worth building until the applicant-driven model is proven.

Everything else from this conversation — paid-since-launch tiers, escrow on accept, the LinkedIn-style profile, and the full security checklist — is in scope for this MVP build.

Update: launch-ready scope expansion

This section is additive. It does not ask Claude Code to modify anything built in Phases 0-7. Everything here is new migrations and new features layered on top of the existing repo, run as a new Phase 8 onward.

Legal flag before building this

Superseded: the MIPS-escrow design in this section (8a-8d below) assumed outside companies and cross-party custody. Given the project is Mauritius-only (companies, talent, and lockedinn all local), the final decision is a direct-pay model with no custody at all — see "Final payment architecture" near the end of this doc. Self-serve company accounts (8a) and the company/talent bank-details tables (8d) are still relevant; the MIPS escrow mechanics in 8b are not.

Once outside companies pay money into lockedinn's own account before it reaches talent, the platform is functioning like a payment intermediary, not just a SaaS charging subscriptions. Get a direct answer from a Mauritius payments and fintech lawyer on whether this requires a payment service license or a specific escrow or trust structure, before the MIPS escrow integration described below goes live. Building the schema and UI now is fine; taking real third-party money through it before that legal question is answered is the risk. Also flag for the same review: AML and KYB obligations on companies now that money flows through the platform's account between unrelated parties, and whether a separate client-funds bank account, distinct from the operating account, is advisable.

8a. Self-serve company accounts and profiles

Companies create their own accounts and profiles instead of being admin-only records. This replaces admin-managed company creation going forward; the 4 existing companies become the first self-serve profiles, claimed by the founder, rather than being recreated.

flowchart TD

```
A[Company signs up] --> B[Company onboarding: name, logo, description, website]
```

```
B --> C[Add bank and beneficiary details for refunds]
```

```
C --> D[New company: capped at small job values]
```

```
D --> E[Post jobs, accept applications]
```

```
E --> F[Clean track record]
```

```
F --> G[Cap lifted after manual admin review]
```

Schema: add `owner_user_id` and `verification_tier` (`new`, `trusted`) to `companies`. Add a `company_bank_details` table (`company_id`, `beneficiary_name`, `bank_name`, `account_number`), used for dispute refunds, not for routine payouts. New companies default to `new` with the job-value cap already planned in the escrow trust guardrails; admin can promote a company to `trusted` after reviewing its track record.

Plan first. This is additive: do not modify existing admin company-management code from Phase 2. Build:

- Migration: add `owner_user_id`, `verification_tier` ('new'/'trusted' default 'new') to `companies`. New table `company_bank_details` (`company_id`, `beneficiary_name`, `bank_name`, `account_number`), RLS: owner and admin only.
- Company signup flow: a new account type 'company' alongside talent/admin, with its own onboarding (name, logo, description, website, bank and beneficiary details).
- /company area: a dashboard for the company to post jobs (reusing the Phase 2 job schema), view applications, and review submissions, mirroring what admin could already do for admin-owned companies, but scoped to the company's own `owner_user_id` via RLS.
- A public company profile page, viewable by talent.
- New companies (`verification_tier` = 'new') can only post jobs under a configurable cap, default \$20. Admin can promote a company to 'trusted' from the admin panel to lift the cap.
- Do not touch the existing admin company CRUD from Phase 2; it continues to work unchanged for admin-owned companies.

8b. Escrow via MIPS for non-owned companies

When a company the founder doesn't own accepts a Talent's application, payment is charged through MIPS into lockedinnn's account and held until the submission is resolved. This is Fix 1 from earlier in this doc, now tied to a specific processor.

flowchart TD

```

A[Company accepts application] --> B[MIPS charges company]
B --> C[Funds land in lockedinnn account, escrow_status=held]
C --> D[Talent submits proof]
D --> E{Company review}
E -- Approved --> F[escrow_status=released; payout row created for Talent]
E -- Rejected --> G[escrow_status stays held; submission=disputed]
G --> H[Admin arbitrates]
H -- Talent wins --> F
H -- Company wins --> I[escrow_status=refunded via MIPS]
```

Plan first. This is additive, for non-admin-owned companies only; admin-owned company jobs keep working exactly as in Phase 3, unchanged. Build:

- Migration: add `escrow_status` ('pending'/'held'/'released'/'refunded') to applications; add `disputed` to the submissions status enum; new disputes table (`submission_id`, `talent_response`, `admin_decision`, `decided_by`, `decided_at`).
- When a non-owned company accepts an application: charge via MIPS, set `escrow_status`='held' only on a successful charge. If the charge fails, the acceptance does not go through; show the company a clear error.
- Route `/api/webhooks/mips` verifying MIPS's signature, idempotent on transaction id.
- Approval (as in Phase 3) now additionally sets `escrow_status`='released' for non-owned-company jobs before creating the payout row.
- Rejection on a non-owned-company job sets submission status to 'disputed' instead of 'rejected' directly, and notifies both sides. Talent has 48 hours to respond before admin arbitration.
- `/admin/disputes` queue: view the submission, `proof_instructions`, talent's response, and the company's stated reason; decide for talent (release) or company (refund via MIPS, falling back to `company_bank_details` only if a MIPS refund isn't possible).
- 5-day timeout: if a non-owned company doesn't respond to a submission, auto-approve and release escrow.
- File-based submissions (links/files) from non-owned-company jobs show a watermarked or reduced-quality version to the company until escrow releases; store the clean version separately, only exposed after release.

Scope note: admin-owned companies (the 4 launch companies) keep working exactly as built in Phases 2-3, with no charge and no escrow, since the founder is paying himself. This flow only applies to companies where `owner_user_id` is not the founder.

8c. Course payments: MIPS instead of Lemon Squeezy

The Phase 5 courses code already works with Lemon Squeezy; that code stays as-is. This adds MIPS as the active processor without removing the Lemon Squeezy integration, so switching back is easy if needed.

Plan first. Do not remove the existing Lemon Squeezy integration from Phase 5; keep it as a fallback. Build:

- A payment_provider config value ('mips' or 'lemonsqueezy'), currently set to 'mips'.
- A MIPS checkout flow for course purchases, mirroring what the Lemon Squeezy flow already does: passes user id, redirects to MIPS checkout.
- Route /api/webhooks/mips (if not already added for 8b) handling the course-purchase event type, idempotent, inserting course_access and calling notify(), the same way the existing Lemon Squeezy webhook does.
- The Buy button on /app/learn reads payment_provider and routes to whichever processor is active, so switching back to Lemon Squeezy later is a one-line config change, not a rebuild.

Smaller provisions, no new phase needed

Item	Decision	What to actually do
Business email	Not set up yet	Use a personal or free address as the visible support contact for now; swap it in once a proper mailbox is ready. The support email is already just a config value, so no code change is needed.
Wise Business	Deferred	<code>payout_details.method</code> is already a free-form field, not a hard enum, so adding <code>wise</code> later needs no migration, just a new option in the payout-method dropdown. Nothing to build now.
Bunny Stream	Provisioned, not used at launch	<code>lessons.video_id</code> already exists as a nullable column from Phase 5; launch with text-only lessons (leave video_id null) and add Bunny playback later with zero schema change.
Calendly	Confirmed	No change; Calendly stays the booking tool for cold-calling proof, as already planned in the accounts table.

8d. Bank and beneficiary details for companies

Covered by the `company_bank_details` table from 8a: companies provide this at onboarding alongside Talent's existing `payout_details`, used specifically for issuing refunds when a dispute resolves in the company's favor and a MIPS refund to the original

payment method isn't possible. Apply the same 24-hour edit-lock and change notification already planned for Talent payout details to this table as well.

Sequencing and assumptions for the broader list

Agency seats, AI grading, ratings, mobile experience, and push notifications are real scope, but building all of them at once alongside 8a-8d would be a lot to ship simultaneously. Suggested order, with the assumption behind each flagged for correction:

Item	Assumption, flag if wrong	Suggested phase
Ratings	Simple two-way rating, 1-5 plus an optional comment, shown on public profiles after a job completes	Phase 9, once 8a-8d are stable
Agency seats	As originally scoped: up to 3 seats sharing one subscription and job pool on the Mini-Agency tier	Phase 9, alongside ratings
Mobile experience	A fully responsive mobile web app, already a build principle from Phase 1, not a native iOS/Android app; flag now if native is actually wanted, since that is a separate, much larger project	No new phase needed unless native is wanted
Push notifications	Web push via the browser Push API and a service worker, not native app push notifications	Phase 10
AI grading	AI-assisted pre-grading: an AI pass suggests pass or fail and draft feedback on course assignments, with admin confirming before it is final, not fully autonomous grading	Phase 10, after manual grading from Phase 6 has enough real submissions to know what good feedback looks like

Recommended order overall: 8a through 8d first, since self-serve companies and escrow are the core of this update and the legally sensitive part, then Phase 9 for ratings and agency seats, then Phase 10 for push notifications and AI-assisted grading.

Final payment architecture: direct pay via Juice QR

Final decision, replacing the MIPS-escrow design above. Since every company and every talent is in Mauritius, lockedinnn never needs to hold job payment at all.

The flow:

```

Job posted -> Talent applies -> Company approves application (no money)
-> Talent submits watermarked/staged proof
-> Company approves submission
-> App displays talent's Juice QR code
-> Company scans it in their own Juice app, confirms amount
-> Talent confirms receipt -> clean file unlocks automatically
-> Job marked complete, rating prompt sent
-> lockedinnn separately charges its own platform fee via MIPS (ordinary
billing, not job money)

```

Why this is the right call: no escrow, no custody, no payment-licensing question, no manual batch queue, no Stripe/US LLC dependency. The staged-reveal mechanic (watermark/preview before payment, clean file after) does the trust work escrow was meant to do, without lockedinnn ever touching third-party money.

Schema:

- `profiles`: add `juice_qr_url` (or equivalent merchant QR reference) for talent, collected during payout-details onboarding alongside existing `payout_details`.
- `submissions`: add `payment_confirmed_by_talent` (boolean) and `payment_confirmed_at` — this is the talent's one-tap action that unlocks the clean file, replacing any escrow-release trigger.
- `jobs`: add `proof_type` (`file_watermarked`, `file_staging`, `verified_event`), required at job creation, drawn only from categories with an approved proof mechanism (see Job categories and proof types, below).

Onboarding note, resolved: this no longer depends on MCB's merchant QR product at all — see the implementation detail below. No MCB confirmation needed before building this.

Implementation detail: lockedinnn-hosted payment page, not a Juice merchant QR

The QR doesn't need to be MCB's own merchant QR format at all. lockedinnn generates its own QR, encoding a link to a page it controls:

QR content: `https://lockedinnn.mu/p/sarah/kx7m2pQf9z...`

Page shows:

Pay Sarah Doe

MCB – Account ending ****6789

Amount: Rs 5,000

Reference: JOB-1847

[Open MCB Juice] <- opens the app; company enters details manually, since no confirmed Juice deep-link prefill exists
Full account number shown separately, below the masked confirmation line, since the company needs the real number to actually complete the transfer

This removes the MCB merchant-QR dependency and onboarding friction entirely — any talent with a bank account or Juice number can be paid this way, no special merchant status required.

Why the token design matters: the random string after the username (`kx7m2pQf9z...`) is the actual security boundary, not the username in the URL. It must be:

- Long and cryptographically random (not a sequential or short id) — this is what stops someone from guessing or enumerating other talent's payment pages.
- Generated fresh per payment request, not reused across jobs, created only at the moment a company approves a submission (matching the existing "QR only revealed after approval" rule).
- Tied to one specific `application_id` /submission, so the page can show exactly that job's amount and reference, nothing generic.
- Time-limited or state-aware: once talent confirms receipt, the same link should flip to showing "this job is already paid" instead of live payment details, rather than staying open indefinitely or 404ing.
- Rate-limited on lookup, so the token can't be brute-forced by hitting the endpoint repeatedly.

Schema: new `payment_requests` table — `id` (the token), `application_id`, `amount_cents`, `reference_code` (short, human-typeable, e.g. JOB-1847, separate from the secure token), `status` (`pending`, `viewed`, `confirmed`, `expired`), `created_at`, `viewed_at`. The masked account number is derived for display; the full number is read from `payout_details` only on this page, never embedded in the QR itself.

Plan first. Build:

- Migration: payment_requests table as above, RLS: readable by the company on that job and the talent who owns it; written only by the server.
- On submission approval: generate a payment_requests row with a cryptographically random id (nanoid or similar, 20+ chars), the job's amount, and a short reference_code derived from the job id.
- Public route /p/[username]/[token]: looks up payment_requests by token (not by username, which is cosmetic only), rate-limited, logs the view (sets viewed_at, increments a view counter). Renders talent's name, bank name, masked account (last 4 digits), the full account number in a clearly separate section, amount, and reference_code. If status is 'confirmed', show a 'this job is already paid' state instead of payment details.
- Generate the QR image (pointing at this URL) server-side when the payment_requests row is created, displayed in-app on the job's payment screen, fetched fresh on every render, never cached client-side.
- Talent's existing 'confirm receipt' action marks the payment_requests row 'confirmed' in the same transaction that unlocks the clean file.

Payment confirmation and verification (final rules)

With direct payment there is no API confirming that money moved, so verification is evidence-based. This section supersedes the earlier one-tap "talent confirms receipt" idea and any auto-confirm-on-silence rule. Core principle: silence never releases the clean file, because the cost of a delay is small and the cost of releasing to a fake payer is real lost work.

flowchart TD

```

A[Company approves submission] --> B[Payment request created, reference JOB-1847]
B --> C[Company pays in Juice, reports it with transaction ID and proof]
C --> D{Talent response within 48h}
D -- Confirms --> E[Payment settled, clean file unlocks]
D -- Disputes --> F[Admin verification]
D -- Silent --> F
F --> G[Company uploads bank statement line]
G --> H{ID, amount, date, reference all match}
H -- Yes --> I[Payment proven, file unlocks]
H -- No --> J[Claim fails, file stays locked, strike issued]

```

Rules:

Situation	What happens
Talent confirms	File unlocks immediately; payment settled
Talent disputes (cannot find the transaction ID)	Admin verification; both sides asked for statement lines
Talent silent for 48h	File stays locked; company must upload a bank-issued statement line; talent is re-notified on every channel
Statement matches ID, amount, date and reference	Admin releases the file even if talent never replies; talent marked unresponsive
Statement missing or does not match	Claim fails, file stays locked, strike on the company
Talent falsely claims non-receipt	Company's statement proves otherwise; strike on the talent

Automated fraud checks (run before any human sees it):

- `company_txn_id` has a unique constraint across all payment requests, so a reused transaction ID is rejected.
- A hash of every proof image is stored; the same image submitted twice is flagged.
- The transaction date must fall after the submission approval time, and the claimed amount must equal `amount_cents` exactly.
- The reference code must appear in the proof; missing reference triggers a flag, not a block.

Notifications to talent when a company reports payment: in-app, email with a one-tap confirm or dispute link, and WhatsApp or SMS if available; reminder at 24h; escalation to admin at 48h.

Admin queue (exceptions only): expected amount vs claimed, transaction ID, reference, proof, statement uploads, countdown timers, and one-click actions: release file, uphold talent, request statements, issue strike. Every action is written to an append-only `payment_events` log.

Strikes: a hidden per-user counter of lost payment disputes. Repeated strikes suspend the account; a company with a confirmed false payment report cannot approve submissions until reviewed. New companies stay capped at small job values, which bounds the cost of any first-time fraud attempt.

Schema additions to `payment_requests`: `company_txn_id` (unique), `company_proof_path`, `company_proof_hash`, `company_marked_paid_at`, `talent_response` (`confirmed`, `disputed`, `null`), `talent_txn_id_entered`, `talent_responded_at`, `statement_path`, `resolution` (`settled`, `proven`, `failed`), `resolved_by`, `resolved_at`. New table `payment_events` (append-only): `payment_request_id`, `actor_id`, `event_type`, `payload`, `created_at`. New `user_strikes` table: `user_id`, `reason`, `payment_request_id`, `created_at`.

Plan first. Additive; do not modify Phases 0-7. Build:

- Migration: the `payment_requests` additions above, plus `payment_events` (append-only: RLS allows insert only, no update or delete) and `user_strikes`. `company_txn_id` gets a unique constraint.
- Company flow: after approving a submission and paying in Juice, a 'Report payment' form requiring transaction ID and a proof image. Server-side checks: unique txn id, proof image hash not seen before, amount equals `amount_cents`, date plausibility. Failures are flagged to admin, not silently accepted.
- Talent flow: notify via in-app and email (and a WhatsApp/SMS hook if configured) with a one-tap confirm or dispute link (signed, single-use token). Confirm unlocks the clean file in the same transaction.
- Scheduled job: at 24h send a reminder; at 48h with no response, set the request to 'awaiting_statement', keep the file locked, and ask the company for a bank statement line upload.
- `/admin/payments` exceptions queue showing expected vs claimed amount, txn id, reference, proof, statement, timers; actions: release file, uphold talent, request statements, issue strike. Every action writes a `payment_events` row.
- Strikes: issuing one increments the user's count; at a configurable threshold suspend the account and notify admin.
- Silence must NEVER auto-release the clean file under any code path. Add a test that proves this.

Job categories and proof types

A job category is only allowed in the job-creation flow if it declares a `proof_type` with a real verification or preview mechanism. This stays admin-gated even after self-serve companies launch (8a): a company proposing a new category that doesn't map to an existing proof type is blocked with a clear message, not silently allowed through.

Category	proof_type	Launch-ready?
Clipping	<code>file_watermarked</code>	Yes

Category	proof_type	Launch-ready?
Content creation / graphic design	file_watermarked	Yes
Copywriting	file_watermarked (preview = partial text)	Yes
Web dev	file_staging	Yes
Cold calling	verified_event (Calendly webhook)	Yes
Social growth / engagement jobs	none yet	No — screenshots are fakeable; needs an OAuth-connected analytics check before it's trustworthy
Live chat / VA / support work	none yet	No — no independent third party like Calendly confirms it happened; needs a logged, timestamped interaction record first
Retainer / ongoing work	none yet	No — breaks the single-submission, single-approval model; would need restructuring into milestone jobs
Pure advice/consulting	none yet	No — too subjective for checkable proof_instructions without a mandatory structured deliverable format
Physical-presence jobs	none yet	No — no digital verification path exists yet

All four launch companies' job types map cleanly onto the first five rows, which is why the MVP can launch on this model without needing any of the unresolved categories.

Expanded category list, all launching day one

All of the categories below ship with the MVP launch, not phased in later. Each maps to an existing proof type, so none of them require new verification infrastructure beyond what's already planned above.

More under file_watermarked :

Category	Watermarked preview
Voiceover / audio narration	Low-quality or truncated clip
Podcast editing	Same mechanism as clipping
Logo / thumbnail design	Watermarked image overlay
Presentation / deck creation	Watermarked PDF export, full file withheld
Photo editing / retouching	Lower-resolution preview
Resume / CV writing	First section visible, rest redacted
Proofreading / document editing	Tracked-changes summary, no clean file
Transcription	First portion of the transcript only
Translation	Partial excerpt
UI/UX mockup design	Low-res export, full source file withheld
Illustration / digital art	Watermarked image
Blog / SEO writing	First paragraphs visible, rest blurred

More under **file_staging** :

Category	What stays withheld
Landing page builds	Source code, hosting access
Mobile app prototypes	Build files, signing credentials
Automation / bot builds	Final handover and credentials
Chrome extension development	Packaged extension file, store access

More under **verified_event** :

Category	Verification source
Appointment setting (any industry)	Calendly, same mechanism as cold calling
Webinar / event hosting	The webinar platform's attendance report
Customer interview scheduling	Calendly webhook

New proof type: `verified_publish` — for jobs where the deliverable is posting content somewhere, verified via that platform's own API confirming the post exists and is public, not a screenshot:

Category	Verified via
UGC / creator posting	Target platform's API
Affiliate / promotional posting	Same
Review / testimonial posting	Same

This is distinct from social-growth/engagement jobs (still excluded): "did this get posted" is objectively checkable through an API; "did this get 1,000 followers" is not, without a separate OAuth-analytics integration that isn't built yet.

Schema update: `jobs.proof_type` enum becomes `file_watermarked`, `file_staging`, `verified_event`, `verified_publish`. Each new category added above must be seeded with its `proof_type` at launch, following the same admin-gating rule: no category goes live without one.

Update: applicant profile view for companies

When a company reviews applications, it can open each applicant's full profile: portfolio, badges, certificates, work history, and stats. This covers applicants only. Browsing all talent and inviting people directly stays deferred, as noted above.

flowchart TD

```

A[Company opens applications for a job] --> B[Applicant cards: photo,
headline, badges, stats, pitch]
B --> C[Open full profile]
C --> D{Active application to this company}
D -- No --> E[Access denied]
D -- Yes --> F[Tabs: overview, portfolio, badges and certificates, work
history, reviews]
F --> G{Decision}
G -- Accept --> H[Chat unlocks, talent notified]
G -- Reject --> I[Talent notified, profile access revoked]
```

What the company sees:

Section	Shown to a company reviewing an application	Notes
Photo, name, headline, bio	Yes	
Badges	Yes	Course name and date earned
Certificates	Yes	Platform-issued marked Verified with a check link; self-uploaded marked Unverified
Portfolio	Yes	View-only watermarked previews; items the talent marked private are hidden
Work history	Yes	Company, job title, completion date; never pay amounts
Stats	Yes	Jobs completed, approval rate, member since; average rating once ratings ship
Phone, email, date of birth, address	Never	Excluded at the query level, not just hidden in the UI
Payout and bank details	Never	Appear only on the payment page after a submission is approved
Strikes and dispute history	Never	Admin-only; companies see approval rate instead
Chat	After acceptance only	Prevents taking the deal off-platform before acceptance

Certificates:

- Platform certificates are issued automatically when a talent passes a course assignment, in the same transaction that awards the badge. Each has a unique random `verify_token`, and a public page `/verify/[token]` shows only the holder's name, certificate title, issue date, and status (valid or revoked).
- External certificates are uploaded by the talent, shown as Unverified, and an admin can mark one verified after checking it.

- Schema: new `certificates` table with `user_id`, `kind` (`platform`, `external`), `title`, `issuer`, `issued_at`, `course_id` (nullable), `verify_token` (unique, nullable), `file_path` (nullable), `status` (`valid`, `revoked`, `pending`), `verified_by`, `verified_at`.
- `portfolio_items` additions: `visibility` (`companies`, `private`, `default companies`) and `preview_path` (the watermarked version), kept separate from the original file path.

Access and security rules:

- A company can read a talent's profile data only while that talent has an application with status pending or accepted on one of that company's jobs. Access is revoked when the application is rejected or withdrawn. Enforce this in RLS through a security-definer function, never only in the UI.
- Talent sees a notice when applying: companies you apply to can view your profile, portfolio, badges, and certificates. The consent time is stored on the application.
- Portfolio media reaches companies only as watermarked previews through short-lived signed URLs, generated by the same background job queue used for submissions. Originals are never exposed.
- Every company profile view is logged in a `profile_views` table (company, talent, application, timestamp).
- Add to the cross-tenant test suite: Company A cannot open the profile of a talent who applied only to Company B, and access ends after rejection.

Plan first. Additive; do not modify Phases 0-7 or the payment code. Build:

- Migration: `certificates` table as described; `portfolio_items` additions (`visibility`, `preview_path`); `profile_views` log; applications. `profile_consent_at`.
- SQL security-definer function `company_can_view_talent(talent_id)`: true only when the calling user owns a company with a job that has an application from that talent with status pending or accepted. Use it in RLS for company reads of profiles, `portfolio_items`, `user_skills`, `certificates` and work history. Admins bypass via `is_admin()`.
- A company-facing profile read model (a view or server function) that omits phone, email, `date_of_birth`, address, `payout_details` and strikes at the query level, so they can never reach the response.
- Applications review screen for /company and /admin: applicant cards (photo, name, headline, top badges, jobs completed, approval rate, rating, pitch preview) with a View profile action that opens a drawer or page with tabs: Overview, Portfolio, Badges and certificates, Work history, Reviews. Accept and Reject actions stay on that screen.
- Certificate issuance: when a course assignment is passed, in the same transaction as the badge award, insert a platform certificate with a

cryptographically random `verify_token`. Public route `/verify/[token]` shows only holder name, title, issue date and status.

- Talent settings: upload external certificates (marked Unverified) and mark portfolio items private. Admin action to mark an external certificate verified.
- Portfolio uploads generate a watermarked preview via the existing background job queue; companies get only previews via short-lived signed URLs.
- Consent notice on the Apply action; store `profile_consent_at`.
- Log every company profile view to `profile_views`.
- Tests: a company cannot read a talent who applied only to another company; access ends after rejection or withdrawal; contact and payout fields are absent from company responses; the verify page exposes only name, title, date and status.

Update: job assets, links, and watermarking

Companies can attach files and links to a job for the talent to work from. Watermarks apply only to what the talent delivers, never to what the company provides, because talent needs source files at full quality.

Deliverables: for `file_watermarked` jobs the finished work must be uploaded to `lockedinn` so the server can generate the watermarked preview; a link is never accepted as the deliverable, since it would let the company download the full-quality file before paying. Links are accepted as deliverables only where the proof type calls for them.

proof_type	Deliverable	Links accepted
<code>file_watermarked</code>	Uploaded file	None
<code>file_staging</code>	Platform-hosted preview on an expiring sandbox, source withheld	Platform-hosted preview only
<code>verified_publish</code>	Public post URL	Target platform only, confirmed by its API
<code>verified_event</code>	Calendly event	Calendly only

Jobs that require editing inside the client's own Drive or document cannot use `file_watermarked`; talent works in their own file and uploads an export.

Source and reference assets from the company: uploads are the default. Links are allowed from an allowlist, each with a role of `source` (needed to do the work) or `reference` (inspiration only).

Category	Allowed domains	Roles
Cloud storage	Google Drive, Dropbox, OneDrive/SharePoint	source
Video	YouTube (including unlisted), Vimeo, Loom	source, reference
Design and docs	Figma, Canva, Google Docs/Sheets/Slides, Notion	source, reference
Social posts	TikTok, Instagram, X, YouTube posts	reference only

Blocked: URL shorteners, messaging and community invites (WhatsApp, Telegram, Discord), non-HTTPS links, raw IP addresses, expiring transfer links such as WeTransfer, and anything not on the allowlist. Admin can add domains. The allowlist is re-checked after redirects, so a permitted domain cannot redirect to a blocked one.

Safeguards:

- Source assets (files and links) are visible only to the talent whose application is accepted, enforced through RLS and short-lived signed URLs. Applicants see the description and reference assets only.
- Links are validated when posted: reachable, publicly accessible, not behind a login wall, and clean on a Google Safe Browsing check. The company is told how to fix a failing link.
- A talent can tap "I can't open this"; the company is notified and has 24 hours to fix it while the deadline pauses. The report is stored as dispute evidence.
- The brief is versioned. Acceptance records the version the talent agreed to. Any later change to the description or assets creates a new version that the talent must acknowledge, and the deadline adjusts.
- A link's contents cannot be frozen, so the Terms should require source material to remain unchanged after acceptance, and the posting form should recommend uploads for anything that must stay fixed.
- Links display their real domain, open through a leaving-lockedinnn notice, in a new tab with `noopener noreferrer`.
- Uploaded files are malware-scanned in the background, restricted by type and size, and deleted from storage a configurable number of days after job completion (default 60).

Schema:

- `job_assets`: `job_id`, `kind` (`file`, `link`), `role` (`source`, `reference`), `label`, `storage_path`, `url`, `domain`, `mime_type`, `size_bytes`, `link_status` (`ok`, `inaccessible`, `blocked`, `unchecked`), `link_checked_at`, `scan_status` (`pending`, `clean`, `infected`).

- `job_brief_versions`: `job_id`, `version`, `description_snapshot`, `assets_snapshot` (json), `created_at`, `acknowledged_at`.
- `link_domain_allowlist`: `domain`, `category`, `allowed_roles`, `notes`.
- `link_reports`: `job_asset_id`, `reported_by`, `created_at`, `resolved_at`.
- `applications`: add `brief_version_id`.

Plan first. Additive; do not modify Phases 0-7 or the payment code. Build:

- Migration: `job_assets`, `job_brief_versions`, `link_domain_allowlist` (seed with the domains above), `link_reports`, and `applications.brief_version_id`.
- Job posting form (admin and company): an Assets section with file upload (resumable for large files) and Add link. Each asset has a role of Source or Reference. Validate with Zod. File types and size limits are configurable; malware scan runs in the background job queue and its status is shown.
- Server-side link validation: https only; domain must be in the allowlist for that role; reject shorteners, messaging invites and IP hosts; follow redirects and re-check the final domain; confirm the link is reachable and not a login wall; run a Google Safe Browsing check; store `link_status`.
- Visibility: applicants see the description and Reference assets only. Source assets are readable only by the talent whose application is accepted, via RLS and short-lived signed URLs.
- On acceptance: snapshot the brief into `job_brief_versions` and set `applications.brief_version_id`. Later edits to the description or assets create a new version that the talent must acknowledge; the deadline adjusts.
- Talent 'I can't open this' button on a link: creates a `link_reports` row, notifies the company, gives 24 hours to fix, pauses the deadline, and is kept as dispute evidence.
- Link display: show the real domain, a leaving-lockedinnn notice, new tab, noopener norereferrer.
- Submission validation: for `file_watermarked` jobs the deliverable accepts a file upload only, never a link. Other proof types accept links only as described in the `proof_type` table. Add a test proving a link cannot be submitted as the deliverable for `file_watermarked`.
- Retention: delete source asset files from storage N days after job completion (default 60, configurable).
- Tests: a non-accepted applicant cannot fetch source assets; blocked domains are rejected including via redirects; the versioning and acknowledgment flow works; the deliverable-link test above.

Delivery leak prevention

A talent-supplied link, or any file sent outside the upload pipeline, lets a company take full-quality work without paying. Until payment is settled, the deliverable must travel only through the upload and watermark pipeline.

Leak route	Rule
Link in the deliverable field	Rejected server-side for <code>file_watermarked</code> jobs
Link in submission notes	External storage and transfer domains blocked in notes until payment settles
Link or attachment in chat from talent	While a payment request is open (from acceptance until settled), talent messages cannot contain storage or transfer links or file attachments; blocked with an explanation. Company messages are unaffected
Off-platform delivery (email, messaging apps)	Cannot be prevented technically. The Terms state that delivering outside the Platform before payment is confirmed forfeits protection, and a company requesting it is breaching the anti-circumvention rule. A "company asked me to deliver off-platform" button opens a strike review
Very large files	Resumable uploads with per-category size caps; the platform stores the original and generates the preview
Preview good enough to use	Preview specification below

Preview specification:

Type	Preview
Video	480p, low bitrate, tiled diagonal semi-transparent watermark across the full frame, not a corner
Audio	Low-bitrate mono with a spoken or tone tag roughly every 10 seconds
Images	Maximum about 800px wide, heavy compression, tiled watermark across the whole image

Type	Preview
Documents and PDFs	Rasterized pages with watermark, or first portion only; copy and paste disabled
Written copy	First 25 percent shown, remainder blurred

Every watermark carries the company's name and the job reference (for example "lockedinnn preview, Company Name, JOB-1847"), so a leaked preview is traceable to the account that received it.

Plan first. Additive; do not modify Phases 0-7 or the payment code. Build:

- Message pipeline rule: while a job's payment request is open (from application accepted until resolution is settled or proven), reject talent-sent messages that contain URLs on storage or transfer domains (Drive, Dropbox, OneDrive, WeTransfer and similar) or any file attachment. Show a clear explanation: deliver through the Submit button, files sent in chat are not protected. Company-sent messages are unaffected. After settlement, lift the restriction.
- Submission notes field: block the same domains for file_watermarked jobs until settlement.
- Preview generation in the background job queue per the preview specification, with a watermark that includes the company name and job reference.
- A 'Company asked me to deliver off-platform' button on the job page that records a report and opens a strike review in /admin/payments.
- Resumable uploads for large deliverables with per-category size caps.
- Tests: a talent message with a Drive link is rejected while payment is open and accepted after settlement; a link in notes is rejected on file_watermarked jobs; the watermark text contains the company name and job reference.

Update: job posting flow with assets

Consolidates the job posting experience for both admin (for admin-owned companies) and self-serve companies. Assets are a step in the flow, built on the `job_assets` design above.

Step	What the company does	Rules
1. Basics	Title, category, description	Only approved categories are selectable; the category sets <code>proof_type</code> automatically

Step	What the company does	Rules
2. Requirements	Required badge or open to all, slots, deadline	
3. Pay	Fixed or per unit, amount	New companies capped at small job values until trusted
4. Proof instructions	Edit a pre-filled checklist for the category	Required and checkable
5. Assets	Upload files or add links, each marked Source or Reference	Limits below
6. Review and publish	Preview of what talent will see	Publish blocked until checks pass

Each step saves to a draft so nothing is lost.

Asset limits (defaults, configurable):

Limit	Default
Assets per job	20
Total size per job	10 GB
Image	20 MB each
Document	50 MB each
Audio	500 MB each
Video	5 GB each, resumable upload
Zip	500 MB each, scanned
Storage quota per company	Lower for new companies, higher for trusted

Allowed types: jpg, png, webp, mp4, mov, webm, mp3, wav, m4a, pdf, docx, xlsx, pptx, txt, and zip. File type is checked from the file's contents (magic bytes), not only the extension. Executables, scripts, and installers are rejected.

Who sees what:

Audience	Sees
Visitors and applicants	Description, proof instructions, Reference assets, and a count of Source files

Audience	Sees
Accepted talent	Everything above plus Source assets and the current brief version
Company and admin	All assets, with scan and link status

Publish is blocked when: any asset scan is pending or failed, any Source link is inaccessible or blocked, proof instructions are empty, or no category is chosen. The company sees which check is failing.

After publishing: edits before any acceptance are free. After acceptance, edits create a new brief version the talent must acknowledge (see the assets section). Deleting a Source asset after acceptance only hides it; it is kept as evidence until the retention period ends.

Schema additions: `job_assets.deleted_at`, `job_assets.sort_order`, `companies.storage_quota_bytes`, and a `proof_templates` table (`category`, `checklist_text`) holding the pre-filled checklist for each category.

Plan first. Additive; reuse the job schema and the job_assets design from the assets section. Build a multi-step job posting flow used by both /admin (admin-owned companies) and /company:

- Steps: Basics, Requirements, Pay, Proof instructions, Assets, Review and publish. Each step saves to a draft job. Validate every step with Zod.
- Basics: the category list contains approved categories only; choosing one sets `proof_type` automatically.
- Pay: enforce the new-company value cap (`verification_tier = 'new'`).
- Proof instructions: pre-fill from `proof_templates` for the category; the field is required.
- Assets step: drag-and-drop upload with progress and resumable large files, thumbnails, a label field, a Source or Reference toggle, and an Add link input showing validation status badges (ok, inaccessible, blocked, unchecked) and scan status for files. Enforce the limits in the table above and the company's storage quota. Check file type from the file contents, not only the extension; reject executables.
- Review step: a 'how talent will see this' preview split into what applicants see and what accepted talent sees; list any failing publish check.
- Publish gating: block publish if any asset scan is pending or failed, any Source link is inaccessible or blocked, proof instructions are empty, or no category is chosen.
- Editing: free before any acceptance; after acceptance, create a new `job_brief_versions` row requiring talent acknowledgment. Deleting a Source asset after acceptance sets `deleted_at` and hides it, keeping

the file until retention.

- Seed proof_templates with a checklist for every launch category.
- Tests: publish is blocked with a pending or infected asset or an inaccessible Source link; applicants cannot fetch Source assets; the storage quota is enforced; a renamed executable is rejected; edits after acceptance create a new version.